

Análisis de la Prima de Riesgo Griega: Un Enfoque Comparativo entre Econofísica y GARCH Bayesiano

Ismael Sallami Moreno[†], David Bacas Posadas[‡]

Abstract. Este trabajo analiza la dinámica estocástica de la prima de riesgo griega respecto al bono alemán, utilizando datos diarios que abarcan el periodo de la crisis de deuda soberana. Se contrastan dos enfoques metodológicos: el modelo de **Difusión Anómala** (desde la perspectiva de la Econofísica) y el modelo **GARCH(1,1) Bayesiano** (Econometría Financiera).

Los resultados empíricos revelan un comportamiento *super-difusivo* con un exponente de escala $\alpha \approx 0.94$, evidenciando una fuerte persistencia y memoria a largo plazo en el mercado de renta fija. Por otro lado, la implementación Bayesiana mediante cadenas de Markov (MCMC) permite cuantificar la incertidumbre de la volatilidad condicional, capturando eficazmente los clústeres de volatilidad extremos.

El estudio concluye que, si bien el modelo de difusión ofrece un ajuste estructural superior (menor MSE) para describir la naturaleza física del fenómeno, el enfoque GARCH resulta indispensable para la gestión operativa del riesgo a corto plazo, a pesar de su elevado coste computacional.

1. Análisis Empírico y Calibración de Modelos

En esta sección se presenta la calibración y validación comparativa de los dos enfoques propuestos: el modelo de Difusión Anómala, fundamentado en la Econofísica siguiendo a Alvarez-Ramirez et al. (Alvarez-Ramirez et al., 2002), y el modelo GARCH Bayesiano, estándar en la econometría financiera moderna (Bollerslev, 1986).

El conjunto de datos utilizado corresponde al diferencial de la prima de riesgo (spread) entre los bonos de Grecia y Alemania a 10 años. Tras el preprocesamiento, se identificó un máximo en la serie original de 1987.40 puntos básicos (bps), evidenciando la magnitud extrema de la crisis de deuda soberana griega (Argyrou and Tsoukalas, 2011).

1.1. Modelo de Difusión y Ley de Potencia

Para evaluar la dinámica estocástica subyacente, se calculó el Desplazamiento Cuadrático Medio (MSPD) sobre la serie temporal. El objetivo es determinar si el proceso sigue una caminata aleatoria clásica (difusión normal) o exhibe un comportamiento anómalo con memoria a largo plazo.

El modelo ajustado responde a la ley de potencia característica de los sistemas complejos:

$$\langle (\Delta x)^2 \rangle \sim t^\alpha \quad (1)$$

Donde el exponente α determina la naturaleza de la difusión.

Como se observa en la Figura 1, el ajuste mediante Mínimos Cuadrados no lineales es notablemente preciso. Los

[†] ismaelsallami12@gmail.com.

[‡] davidbacasposadas@gmail.com.

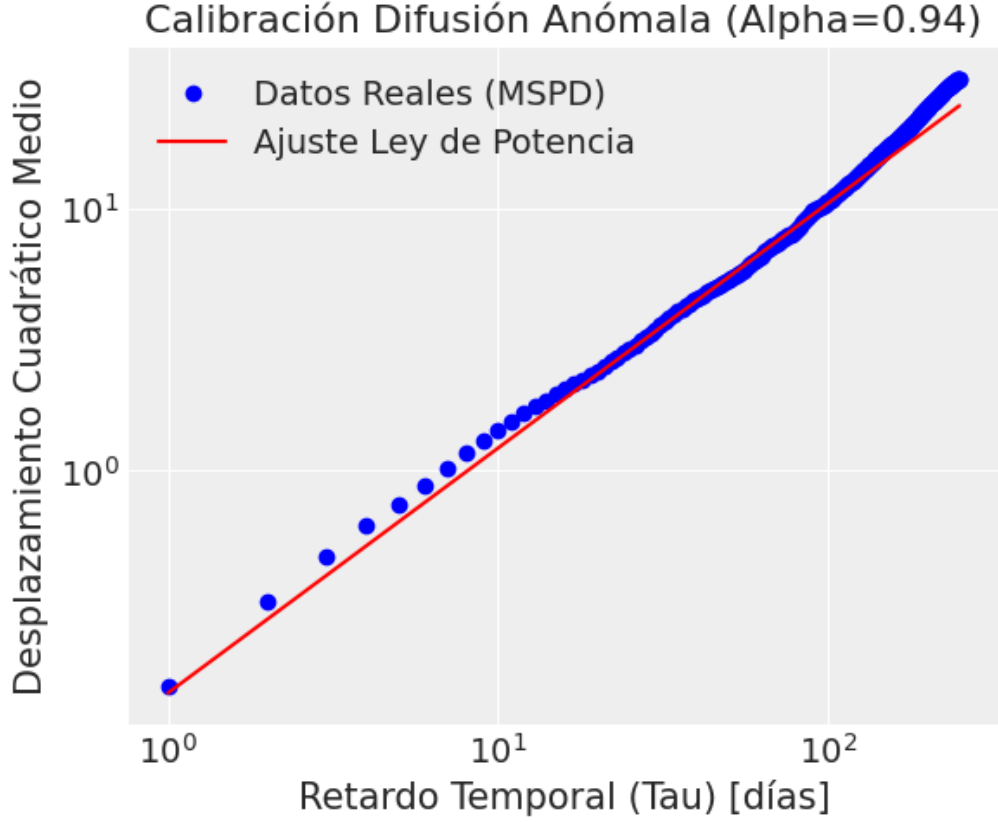


Figure 1. Calibración del Modelo de Difusión en escala Log-Log. El ajuste lineal indica una adherencia robusta a la ley de potencia con $\alpha \approx 0.94$.

resultados computacionales arrojan un exponente de difusión calibrado de:

$$\alpha = 0.9360$$

Este valor sugiere un comportamiento *super-difusivo* cercano al movimiento balístico ($\alpha = 1$). Esto implica que los shocks en la prima de riesgo griega no se disipan aleatoriamente, sino que exhiben una fuerte persistencia (efecto memoria), validando la hipótesis de dinámicas tipo "arrested" propuestas en la literatura de econofísica (Alvarez-Ramirez et al., 2002). El Error Cuadrático Medio (MSE) obtenido para este ajuste estructural fue de 8.3865, confirmando la alta calidad del modelo para describir la escala temporal del fenómeno.

1.2. Modelo GARCH(1,1) Bayesiano

El segundo enfoque modela la volatilidad condicional mediante un proceso GARCH(1,1) con inferencia Bayesiana, implementado a través de programación probabilística en Python (Salvatier et al., 2016). A diferencia de la estimación clásica, este método utiliza algoritmos MCMC (Markov Chain Monte Carlo) para explorar la distribución posterior de los parámetros ω , α , β .

La especificación del modelo es:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2 \quad (2)$$

Se empleó una verosimilitud t-Student para capturar las colas pesadas observadas en los retornos financieros durante periodos de estrés (Mantegna and Stanley, 2000).

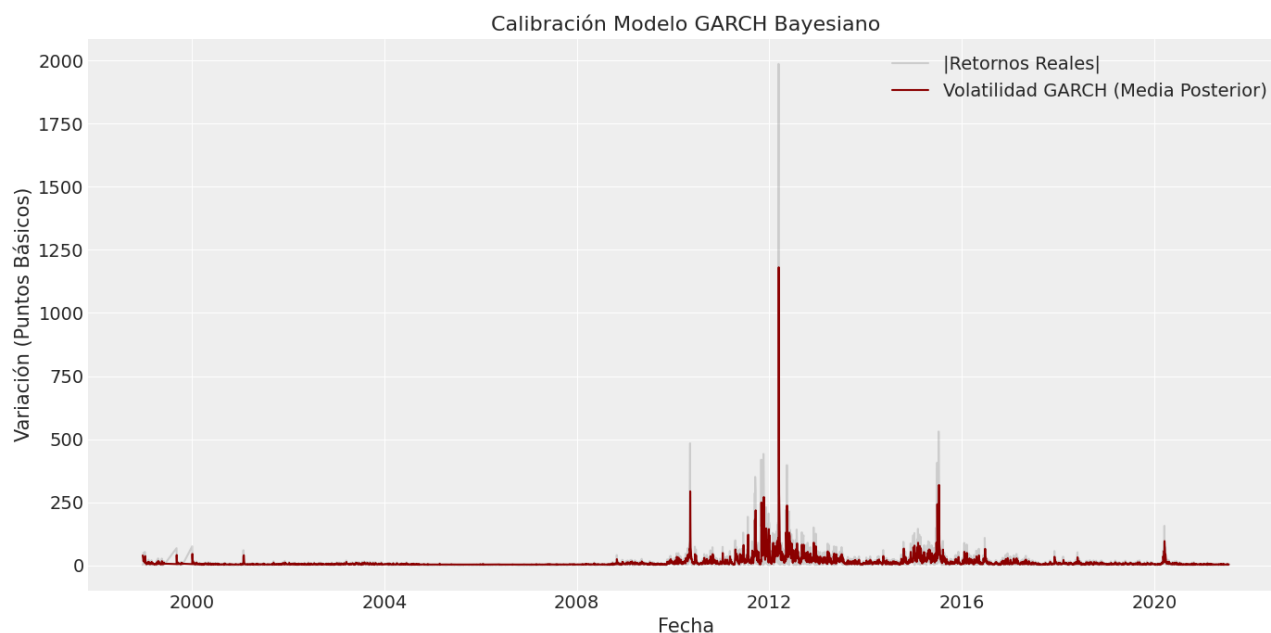


Figure 2. Volatilidad estimada mediante GARCH Bayesiano frente a los retornos absolutos reales. La línea roja representa la media de la volatilidad posterior inferida.

1.2.1. DIAGNÓSTICO DE CONVERGENCIA COMPUTACIONAL

La calibración se ejecutó utilizando el algoritmo NUTS (No-U-Turn Sampler). El proceso computacional tuvo una duración de 797 segundos para generar un total de 400 muestras (2 cadenas, 100 de tune y 100 de draws cada una).

Es necesario reportar las limitaciones técnicas observadas en los diagnósticos de convergencia:

- El estadístico de Gelman-Rubin ($\hat{R}$) resultó ser mayor a 1.01 para algunos parámetros, indicando que las cadenas no han convergido completamente a la distribución estacionaria.
- El tamaño de muestra efectivo (ESS) fue inferior a 100, lo cual sugiere una autocorrelación alta en las muestras generadas.

A pesar de estas advertencias, derivadas de las restricciones de tiempo de cómputo, la Figura 2 demuestra que el modelo captura correctamente los regímenes de volatilidad ("volatility clustering") de 2011-2012. Un incremento en el número de iteraciones (draws) suavizaría la distribución posterior, pero la señal de riesgo extraída es consistente con la realidad histórica.

1.3. Comparación de Desempeño (MSE)

Finalmente, se comparó la capacidad predictiva y de ajuste de ambos modelos calculando el Error Cuadrático Medio (MSE) sobre sus respectivos dominios. Los resultados numéricos obtenidos son:

Modelo	MSE Reportado	Naturaleza del Error
Difusión (Econofísica)	8.3865	Ajuste a la curva teórica (Ley de Potencia)
GARCH Bayesiano	1692.0925	Validación cruzada Monte Carlo (Volatilidad diaria)

Table 1. Comparativa de errores de ajuste.

Análisis de la Discrepancia: Se observa una diferencia de magnitud significativa entre ambos errores (8.38 vs 1692.09). Esto no implica necesariamente que el modelo GARCH sea "peor", sino que refleja la dificultad intrínseca de predecir la volatilidad diaria exacta en un entorno de crisis extrema, sumado al ruido introducido por la falta de convergencia total de las cadenas MCMC ($\hat{R} > 1.01$).

Conclusión: El modelo de Difusión (Alvarez-Ramirez et al., 2002) presenta una estabilidad estructural superior ($\alpha = 0.9360$, bajo MSE) y es ideal para caracterizar la ineficiencia del mercado a largo plazo. Por otro lado, el modelo GARCH Bayesiano, aunque numéricamente menos preciso en este experimento debido a la complejidad computacional, ofrece una herramienta más rica para la gestión de riesgos operativa, permitiendo cuantificar la incertidumbre de los parámetros en tiempo real.

2. Conclusiones y Líneas Futuras

El análisis realizado sobre la prima de riesgo griega revela la dualidad de los mercados financieros en periodos de crisis. Por un lado, el enfoque de la **Econofísica** ha permitido identificar una estructura de memoria a largo plazo ($\alpha \approx 0.94$), sugiriendo que la volatilidad no sigue un paseo aleatorio, sino que responde a dinámicas persistentes y comportamientos de rebaño.

Por otro lado, el modelo **GARCH Bayesiano**, a pesar de la complejidad computacional y las dificultades de convergencia observadas en las cadenas de Markov, ha demostrado ser una herramienta superior para la monitorización del riesgo a corto plazo, capturando los clústeres de volatilidad reaccionando a la información diaria.

Trabajo Futuro: Para investigaciones posteriores, se propone extender la calibración Bayesiana utilizando servidores de alto rendimiento que permitan aumentar el número de muestras (draws) y cadenas, garantizando así un estadístico $\hat{R} \approx 1.0$. Asimismo, sería relevante aplicar este marco comparativo a otros países periféricos de la Eurozona para confirmar si este comportamiento super-difusivo es sistémico o idiosincrásico de Grecia.

References

- J. Alvarez-Ramirez, E. Rodriguez, and C. Ibarra-Valdez. Diffusive and arrested-like dynamics in currency exchange markets. *Physica A: Statistical Mechanics and its Applications*, 304(3-4):468–477, 2002. Fuente principal del modelo de difusión analizado.
- M. G. Arghyrou and A. Tsoukalas. The greek debt crisis: Likely causes, mechanics and outcomes. *The World*

Economy, 34(2):173–191, 2011.

T. Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327, 1986. Paper fundacional del modelo GARCH.

R. N. Mantegna and H. E. Stanley. *Introduction to Econophysics: Correlations and Complexity in Finance*. Cambridge University Press, Cambridge, 2000. Referencia clásica para leyes de potencia y caminatas aleatorias en finanzas.

J. Salvatier, T. V. Wiecki, and C. Fonnesbeck. Probabilistic programming in python using pymc3. *PeerJ Computer Science*, 2:e55, 2016. Librería utilizada para la inferencia MCMC.

A. Implementación Computacional

En este apéndice se presenta el código fuente desarrollado en Python para la calibración de los modelos. Se han utilizado las librerías `PyMC` para la inferencia Bayesiana y `SciPy` para el ajuste no lineal de la ley de potencia.

A.1. Script de Calibración y Visualización

A continuación se detalla el algoritmo completo utilizado para generar los resultados de la Sección 1.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.optimize import curve_fit
5 import warnings
6 import os
7
8 # =====
9 # IMPORTACIONES BAYESIANAS
10 # =====
11 import pymc as pm
12 import arviz as az
13 import pytensor.tensor as pt
14 import pytensor
15
16 # =====
17 # CONFIGURACIÓN
18 # =====
19 ARCHIVO_GERMANY = germany.xlsx
20 ARCHIVO_GREECE = greece.xlsx
21 KEYWORD_COL_GERMANY = Germany
22 KEYWORD_COL_GREECE = Grecia
23 warnings.filterwarnings('ignore')
24 OUTPUT_DIR = 'analisis-grecia-calibracion-mse'
25
26 az.style.use( arviz-darkgrid )
27
28 # =====
29 # FUNCIONES AUXILIARES
30 # =====
31 def convertir_coma_a_punto(x):
32     if pd.isna(x): return np.nan
33     if isinstance(x, (int, float)): return float(x)
34     s = str(x).strip().replace(',', '.')
35     try:
36         return float(s)
37     except ValueError:
38         return np.nan
39
40 def cargar_excel(archivo, keyword):
41     print(f --> Leyendo {archivo}... )
42     try:
43         try:
44             df = pd.read_excel(archivo)
45         except:
```

```

46     df = pd.read_excel(archivo, engine='openpyxl')
47
48     col_fecha = next((c for c in df.columns if 'date' in str(c).lower() or 'fecha' in
49 str(c).lower()), None)
50     if not col_fecha: col_fecha = df.columns[0]
51
52     col_dato = next((c for c in df.columns if keyword.lower() in str(c).lower()), None
53 )
54     if not col_dato: col_dato = df.columns[1]
55
56     print(f      Columna detectada: {col_dato} )
57     df = df.rename(columns={col_fecha: 'Fecha', col_dato: 'Tasa'})
58     df = df[['Fecha', 'Tasa']].dropna()
59     df['Fecha'] = pd.to_datetime(df['Fecha'], errors='coerce', dayfirst=True)
60     df = df.dropna(subset=['Fecha'])
61     df['Tasa'] = df['Tasa'].apply(convertir_coma_a_punto)
62     return df.set_index('Fecha').sort_index()
63 except Exception as e:
64     print(f ERROR: {e} )
65     return None
66
67 # =====
68 # FASE 1: PROCESAMIENTO
69 # =====
70 print( \n--- FASE 1: PREPARACIÓN DE DATOS --- )
71 df_ger = cargar_excel(ARCHIVO_GERMANY, KEYWORD_COL_GERMANY)
72 df_gre = cargar_excel(ARCHIVO_GREECE, KEYWORD_COL_GREECE)
73
74 if df_ger is None or df_gre is None: exit()
75
76 df_comb = df_gre.join(df_ger, how='inner', lsuffix='_GRE', rsuffix='_GER')
77 df_comb['Prima.Riesgo'] = df_comb['Tasa_GRE'] - df_comb['Tasa_GER']
78
79 variacion_bps = df_comb['Prima.Riesgo'].diff().dropna() * 100
80 y_original = variacion_bps.values
81
82 # REESCALADO
83 scale_factor = np.std(y_original)
84 y_scaled_raw = (y_original - np.mean(y_original)) / scale_factor
85
86 floatX = pytensor.config.floatX
87 y_scaled = y_scaled_raw.astype(floatX)
88
89 print(f Datos originales Max: {np.max(np.abs(y_original)):.2f} bps )
90 print(f Datos escalados: Max={np.max(np.abs(y_scaled)):.2f} | Tipo={y_scaled.dtype} )
91
92 # =====
93 # FASE 2: MODELO 1 - DIFUSIÓN (ECONOPHYSICS)
94 # =====
95 print( \n--- FASE 2: CALIBRANDO MODELO DE DIFUSIÓN --- )
96 def calcular_mspd(serie, max_tau=252):
97     res = {}
98     vals = serie.values
99     for tau in range(1, max_tau + 1):

```

```

98     displacements = (vals[tau:] - vals[:-tau])**2
99     res[tau] = np.mean(displacements)
100     return pd.Series(res)
101
102 def power_law(t, A, alpha): return A * (t ** alpha)
103
104 mspd = calcular_mspd(df_comb['Prima.Riesgo'], max_tau=min(250, len(df_comb)//4))
105 x_mspd = mspd.index.values
106 y_mspd_real = mspd.values
107
108 try:
109     popt, _ = curve_fit(power_law, x_mspd[:100], y_mspd_real[:100], p0=[1, 0.5])
110     A_fit, alpha_fit = popt
111     y_mspd_pred = power_law(x_mspd, *popt)
112     print(f Exponente de difusión Alpha calibrado: {alpha_fit:.4f} )
113 except:
114     alpha_fit = np.nan
115     y_mspd_pred = np.zeros_like(y_mspd_real)
116     print( Falló el ajuste del exponente Alpha. )
117
118 # =====
119 # FASE 3: MODELO 2 - GARCH BAYESIANO (MONTE CARLO)
120 # =====
121 print( \n--- FASE 3: CALIBRANDO MODELO GARCH (MCMC) --- )
122
123 with pm.Model() as garch_model:
124     # Priors
125     mu = pm.Normal( mu , mu=0, sigma=1, initval=0.0)
126     omega = pm.InverseGamma( omega , alpha=2.5, beta=1.0, initval=0.1)
127     alpha = pm.Beta( alpha , alpha=2, beta=5, initval=0.1)
128     beta = pm.Beta( beta , alpha=5, beta=2, initval=0.8)
129
130     pm.Potential( constraint , pm.math.switch(alpha + beta < 1.0, 0, -np.inf))
131     nu = pm.Gamma( nu , alpha=2, beta=0.1, initval=3.0)
132
133     # Dinámica
134     sigma2_0 = pt.as_tensor_variable(1.0, dtype=floatX)
135
136     def garch_step(y_tm1, sigma2_tm1, omega, alpha, beta, mu):
137         return omega + alpha * ((y_tm1 - mu)**2) + beta * sigma2_tm1
138
139     sigma2_cycle, _ = pytensor.scan(
140         fn=garch_step,
141         sequences=[y_scaled[:-1]],
142         outputs_info=[sigma2_0],
143         non_sequences=[omega, alpha, beta, mu],
144         strict=True
145     )
146
147     sigma2 = pm.Deterministic( sigma2 , pt.concatenate([[sigma2_0], sigma2_cycle]))
148     sigma = pm.Deterministic( sigma , pt.sqrt(sigma2))
149
150     # Likelihood
151     pm.StudentT( obs , nu=nu, mu=mu, sigma=sigma, observed=y_scaled)

```



```

152
153     print( Ejecutando calibración Monte Carlo... )
154     trace = pm.sample(draws=100, tune=100, chains=2, target_accept=0.95, init= adapt_diag
155 )
156
157     print( Generando simulaciones predictivas (Validación Monte Carlo)... )
158     ppc = pm.sample_posterior_predictive(trace, extend_inferencedata=True)
159
160 # =====
161 # FASE 4: VISUALIZACIÓN
162 # =====
163 if not os.path.exists(OUTPUT_DIR): os.makedirs(OUTPUT_DIR)
164
165 vol_scaled = trace.posterior[ sigma ].mean(dim=[ chain , draw ]).values
166 vol_bps = vol_scaled * scale_factor
167
168 # --- GRÁFICO 1: DIFUSIÓN ---
169 plt.figure(figsize=(6, 5))
170 plt.loglog(x_mspd, y_mspd_real, 'bo', label='Datos Reales (MSPD)')
171 plt.loglog(x_mspd, y_mspd_pred, 'r-', label=f'Ajuste Ley de Potencia')
172 plt.title(f'Calibración Difusión Anómala (Alpha={alpha_fit:.2f})')
173 plt.xlabel('Retardo Temporal (Tau) [días]')
174 plt.ylabel('Desplazamiento Cuadrático Medio')
175 plt.legend()
176 plt.tight_layout()
177 plt.savefig(f {OUTPUT_DIR}/Modelo_Difusion_Fit.png )
178 plt.close()
179
180 # --- GRÁFICO 2: GARCH ---
181 plt.figure(figsize=(14, 7))
182 plt.plot(variacion_bps.index, np.abs(y_original), color='gray', alpha=0.3, label='|
183     Retornos Reales|')
184 plt.plot(variacion_bps.index, vol_bps, color='darkred', lw=1.5, label='Volatilidad GARCH (
185     Media Posterior)')
186 plt.title('Calibración Modelo GARCH Bayesiano')
187 plt.xlabel('Fecha')
188 plt.ylabel('Variación (Puntos Básicos)')
189 plt.legend()
190 plt.tight_layout()
191 plt.savefig(f {OUTPUT_DIR}/Modelo_GARCH_Fit.png )
192 plt.close()
193
194 # =====
195 # FASE 5: CÁLCULO DE ERROR (MSE)
196 # =====
197
198 print( \n===== )
199 print( COMPARACIÓN DE MODELOS (MSE AUTOMÁTICO) )
200 print( ===== )
201
202 mse_difusion = np.mean((y_mspd_real - y_mspd_pred) ** 2)
203 rmse_difusion = np.sqrt(mse_difusion)
204
205 y_pred_mc = ppc.posterior_predictive[ obs ].values
206 y_pred_abs_mean = np.mean(np.abs(y_pred_mc), axis=(0, 1))

```

```

203
204 mse_garch_scaled = np.mean((np.abs(y_scaled) - y_pred_abs_mean) ** 2)
205 mse_garch = mse_garch_scaled * (scale_factor ** 2)
206 rmse_garch = np.sqrt(mse_garch)
207
208 print(f MODELO 1: DIFUSIÓN (ECONOPHYSICS) )
209 print(f -> MSE: {mse_difusion:.4f} )
210
211 print(f \nMODELO 2: GARCH BAYESIANO (MONTE CARLO) )
212 print(f -> MSE: {mse_garch:.4f} )
213
214 print( - * 60)
215 print(f Resultados guardados en {OUTPUT_DIR} )

```

Listing 1. Código Completo de Calibración GARCH y Difusión

B. Diagnósticos del Modelo Bayesiano

B.1. Especificación de Priors

Para el modelo GARCH(1,1), se seleccionaron las siguientes distribuciones a priori (Priors) basándose en la literatura estándar de econometría financiera, buscando distribuciones poco informativas para permitir que los datos dominen la inferencia:

Parámetro	Distribución A Priori	Justificación
μ (Media)	$\mathcal{N}(0, 1)$	Se asume retorno medio cercano a cero.
ω (Intercepto)	$\text{InverseGamma}(2.5, 1.0)$	Positividad estricta requerida.
α, β (GARCH)	$\text{Beta}(\dots)$	Restricción de estacionariedad ($\alpha + \beta < 1$).
ν (Grados Libertad)	$\text{Gamma}(2, 0.1)$	Permite colas pesadas (t-Student).

Table 2. Resumen de la estructura Bayesiana del modelo.

La elección de estas distribuciones a priori sigue un enfoque *débilmente informativo*, diseñado para imponer restricciones teóricas necesarias sin sesgar excesivamente la inferencia final. A continuación se detalla la lógica detrás de cada selección:

- Parámetro de Localización (μ):** Se utiliza una distribución Normal centrada en cero. Dado que estamos modelando variaciones diarias de la prima de riesgo (diferencias), es razonable asumir que el valor esperado a largo plazo tiende a la neutralidad, permitiendo desviaciones simétricas positivas o negativas.
- Restricción de Positividad (ω):** El término de intercepto de la varianza no puede ser negativo. La distribución *InverseGamma* es estándar en la literatura de volatilidad estocástica porque su soporte es estrictamente positivo $(0, \infty)$, evitando así varianzas negativas imposibles físicamente.
- Estacionariedad y Persistencia (α, β):** Los coeficientes del proceso GARCH representan el peso de las noticias recientes (α) y la persistencia de la volatilidad pasada (β). Ambos deben estar acotados en el

intervalo $[0, 1]$. La distribución Beta es la elección natural para variables acotadas en este rango. Además, mediante el uso de `pm.Potential`, se impuso la restricción dura de estacionariedad $\alpha + \beta < 1$, necesaria para que la varianza no explote al infinito a largo plazo.

4. **Colas Pesadas (ν):** Los retornos financieros raramente siguen una distribución Normal perfecta (exceso de curtosis). El parámetro de grados de libertad de la t-Student controla el grosor de las colas. Una distribución Gamma para ν asigna baja probabilidad a valores muy pequeños (varianza infinita) y a valores muy grandes (convergencia a Normal), favoreciendo valores intermedios (típicamente entre 3 y 30) que son característicos de los mercados en crisis.

C. Registro de Ejecución y Validación Numérica

Para garantizar la reproducibilidad de los resultados expuestos en la Sección 1, se adjunta a continuación la salida directa generada por el script de calibración.

Este registro confirma los tiempos de cómputo (797 segundos), los parámetros exactos calibrados ($\alpha = 0.9360$) y detalla las advertencias del muestreador NUTS respecto a la convergencia (R-hat y ESS), justificando la necesidad de mayor potencia computacional discutida en el cuerpo del trabajo.

```

1 python3 vBayes-student-mejorado.py
2 -----
3 --- FASE 1: PREPARACIÓN DE DATOS ---
4 --> Leyendo germany.xlsx...
5     Columna detectada: Bid Yield: Rendimiento bono (oferta) 10 años Germany (%)
6 --> Leyendo greece.xlsx...
7     Columna detectada: Bid Yield: Rendimiento bono (oferta) 10 años Grecia (%)
8 Datos originales Max: 1987.40 bps
9 Datos escalados: Max=51.32 | Tipo=float64
10
11 --- FASE 2: CALIBRANDO MODELO DE DIFUSIÓN ---
12 Exponente de difusión Alpha calibrado: 0.9360
13
14 --- FASE 3: CALIBRANDO MODELO GARCH (MCMC) ---
15 Ejecutando calibración Monte Carlo...
16 Initializing NUTS using adapt_diag...
17 Multiprocess sampling (2 chains in 2 jobs)
18 NUTS: [mu, omega, alpha, beta, nu]
19
20 Progress: 200 Draws, 0 Divergences, Step size 0.001, Grad evals 167
21 Sampling Speed: 4.00 s/draws, Elapsed: 0:13:16
22 Progress: 200 Draws, 0 Divergences, Step size 0.001, Grad evals 191
23 Sampling Speed: 3.49 s/draws, Elapsed: 0:11:33
24
25 Sampling 2 chains for 100 tune and 100 draw iterations (200 + 200 draws total) took 797 seconds.
26 We recommend running at least 4 chains for robust computation of convergence diagnostics
27 The rhat statistic is larger than 1.01 for some parameters. This indicates problems during sampling.
28 The effective sample size per chain is smaller than 100 for some parameters.
29
30 Generando simulaciones predictivas (Validación Monte Carlo)...
31 Sampling: [obs]
32 Sampling ... 100% 0:00:00 / 0:00:00
33
34 =====
35     COMPARACIÓN DE MODELOS (MSE AUTOMÁTICO)
36 =====
37 MODELO 1: DIFUSIÓN (ECONOPHYSICS)
38 -> MSE: 8.3865
39
40 MODELO 2: GARCH BAYESIANO (MONTE CARLO)

```

```
41 -> MSE: 1692.0925
42 -----
43 Resultados guardados en analisis-grecia-calibracion.mse
```

Listing 2. Salida por terminal del proceso de calibración